

EXPERIMENT 13: ALPHA-BETA PRUNING ALGORITHM FOR GAMING

AIM

To develop a Python program that implements the Alpha-Beta Pruning Algorithm for game playing to reduce the number of nodes evaluated while finding the optimal move.

OBJECTIVE

- To understand the Alpha-Beta Pruning technique.
- To optimize the Minimax search process.
- To eliminate unnecessary branches during game-tree evaluation.
- To determine the optimal move efficiently.

ALGORITHM

Step 1: Start.

Step 2: Construct the game tree.

Step 3: Initialize **Alpha** = $-\infty$ and **Beta** = $+\infty$.

Step 4: Traverse the tree using the Minimax algorithm.

Step 5: Update Alpha at MAX nodes.

Step 6: Update Beta at MIN nodes.

Step 7: If **Alpha** $\geq$ **Beta**, prune the remaining branches.

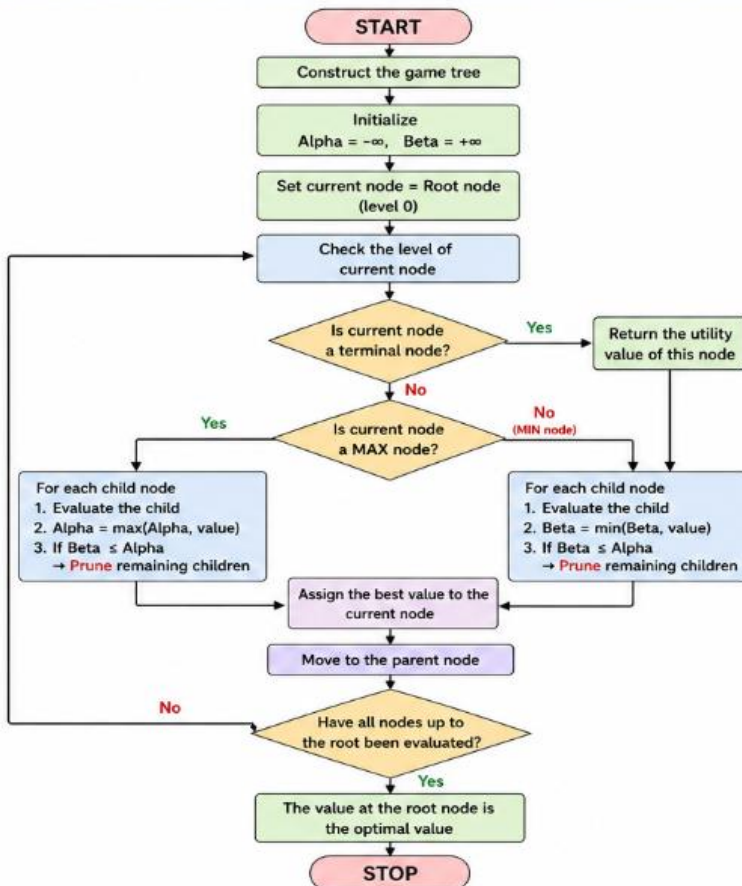
Step 8: Continue until the root node is evaluated.

Step 9: Display the optimal value.

Step 10: Stop.

FLOW CHART

ALPHA-BETA PRUNING ALGORITHM FOR GAMING - FLOWCHART



SOURCE CODE

```
import math
```

```
def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, height):
```

```
    if depth == height:
```

```
        return values[nodeIndex]
```

```
    if maximizingPlayer:
```

```
        best = -math.inf
```

```
        for i in range(2):
```

```
    val = alphabeta(depth + 1, nodeIndex * 2 + i, False,
                    values, alpha, beta, height)
    best = max(best, val)
    alpha = max(alpha, best)

    if beta <= alpha:
        break

    return best

else:
    best = math.inf

    for i in range(2):
        val = alphabeta(depth + 1, nodeIndex * 2 + i, True,
                        values, alpha, beta, height)
        best = min(best, val)
        beta = min(beta, best)

        if beta <= alpha:
            break

    return best

values = [3, 5, 6, 9, 1, 2, 0, -1]
height = 3
```

```
print("Optimal Value:",  
      alphabeta(0, 0, True, values,  
                -math.inf, math.inf, height))
```

OUTPUT

```
Optimal Value: 5
```

RESULT

The Python program successfully implements the Alpha-Beta Pruning Algorithm. It evaluates only the necessary nodes by pruning irrelevant branches, thereby improving the efficiency of the Minimax search while producing the same optimal result.